

Лабораторная работа №4

Выполнила: Сингатуллина Алина Марсовна

Группа 6204-010302D

Оглавление

Задание 1	3
Задание 2	3
Задание 3.....	4
Задание 4.....	5
Задание 5.....	8
Задание 6.....	9
Задание 7.....	9
Задание 8.....	10
Задание 9.....	11

Задание 1

В классах `ArrayTabulatedFunction` и `LinkedListTabulatedFunction` добавила конструкторы, получающие сразу все точки функции в виде массива объектов типа `FunctionPoint`. Если точек задано меньше двух, или если точки в массиве не упорядочены по значению абсциссы, конструкторы выбрасывают исключение `IllegalArgumentException`. При написании конструкторов обеспечивала корректную инкапсуляцию.

```
public ArrayTabulatedFunction(FunctionPoint[] points) {
    if (points.length < 2) {
        throw new IllegalArgumentException("Количество точек не может быть
меньше двух");
    }

    // проверка упорядоченности с машинным эпсилоном
    for (int i = 1; i < points.length; i++) {
        if (points[i].getX() - points[i-1].getX() < -EPSILON) {
            throw new IllegalArgumentException("Точки не упорядочены по
возрастанию X");
        }
    }

    this.pointsCount = points.length;
    this.points = new FunctionPoint[pointsCount + 10];
    for (int i = 0; i < pointsCount; i++) {
        this.points[i] = new FunctionPoint(points[i]);
    }
}
```

```
public LinkedListTabulatedFunction(FunctionPoint[] points) {
    if (points.length < 2) {
        throw new IllegalArgumentException("Количество точек не может быть
меньше двух");
    }

    // проверка упорядоченности
    for (int i = 1; i < points.length; i++) {
        if (points[i].getX() - points[i-1].getX() < -EPSILON) {
            throw new IllegalArgumentException("Точки не упорядочены по
возрастанию X");
        }
    }

    initHead();
    for (FunctionPoint point : points) {
        addNodeToTail().point = new FunctionPoint(point);
    }
    this.pointsCount = points.length;
}
```

Задание 2

В пакете `functions` создала интерфейс `Function`, описывающий функции одной переменной и содержащий следующие методы:

- `public double getLeftDomainBorder()` – возвращает значение левой границы области определения функции;
- `public double getRightDomainBorder()` – возвращает значение правой границы области определения функции;
- `public double getFunctionValue(double x)` – возвращает значение функции в заданной точке.

Исключите соответствующие методы из интерфейса `TabulatedFunction`.

```
package functions;

public interface Function {
    double getLeftDomainBorder();
    double getRightDomainBorder();
    double getFunctionValue(double x);
}
```

Задание 3

Создала пакет `functions.basic` и описала в нем классы аналитических функций:

`Cos.java`

```
package functions.basic;
public class Cos extends TrigonometricFunction {
    @Override
    public double getFunctionValue(double x) {
        return Math.cos(x);
    }
}
```

`Sin.java`

```
package functions.basic;
public class Sin extends TrigonometricFunction {
    @Override
    public double getFunctionValue(double x) {
        return Math.sin(x);
    }
}
```

`Tan.java`

```
package functions.basic;
public class Tan extends TrigonometricFunction {
    @Override
    public double getFunctionValue(double x) {
        return Math.tan(x);
    }
}
```

`Exp.java`

```
package functions.basic;
import functions.Function;

public class Exp implements Function {
    @Override
    public double getLeftDomainBorder() {
        return Double.NEGATIVE_INFINITY;
    }
    @Override
    public double getRightDomainBorder() {
        return Double.POSITIVE_INFINITY;
    }
}
```

```

@Override
public double getFunctionValue(double x) {
    return Math.exp(x);
}
}

```

Log.java

```

package functions.basic;
import functions.Function;
public class Log implements Function {
    private final double base;

    public Log(double base) {
        if (base <= 0 || Math.abs(base - 1) < 1e-10) {
            throw new IllegalArgumentException("Основание логарифма должно
            БЫТЬ > 0 и ≠ 1");
        }
        this.base = base;
    }
    @Override
    public double getLeftDomainBorder() {
        return 0; // логарифм определен при x > 0
    }
    @Override
    public double getRightDomainBorder() {
        return Double.POSITIVE_INFINITY;
    }
    @Override
    public double getFunctionValue(double x) {
        if (x <= 0) {
            return Double.NaN; // log(0) и log(отрицательных) не существует
        }
        return Math.log(x) / Math.log(base);
    }
}

```

TrigonometricFunction.java

```

package functions.basic;

import functions.Function;

public abstract class TrigonometricFunction implements Function {
    @Override
    public double getLeftDomainBorder() {
        return Double.NEGATIVE_INFINITY;
    }

    @Override
    public double getRightDomainBorder() {
        return Double.POSITIVE_INFINITY;
    }
}

```

Задание 4

Создала пакет functions.meta и описала в нем классы комбинирования функций:

Composition.java

```

package functions.meta;
import functions.Function;
public class Composition implements Function {

```

```

private final Function f1, f2;

public Composition(Function f1, Function f2) {
    this.f1 = f1;
    this.f2 = f2;
}

@Override
public double getLeftDomainBorder() {
    return f1.getLeftDomainBorder();
}

@Override
public double getRightDomainBorder() {
    return f1.getRightDomainBorder();
}

@Override
public double getFunctionValue(double x) {
    return f2.getFunctionValue(f1.getFunctionValue(x));
}
}

```

Mult.java

```

package functions.meta;
import functions.Function;
public class Mult implements Function {
    private final Function f1, f2;

    public Mult(Function f1, Function f2) {
        this.f1 = f1;
        this.f2 = f2;
    }

    @Override
    public double getLeftDomainBorder() {
        return Math.max(f1.getLeftDomainBorder(), f2.getLeftDomainBorder());
    }

    @Override
    public double getRightDomainBorder() {
        return Math.min(f1.getRightDomainBorder(),
f2.getRightDomainBorder());
    }

    @Override
    public double getFunctionValue(double x) {
        if (x < getLeftDomainBorder() || x > getRightDomainBorder()) {
            return Double.NaN;
        }
        return f1.getFunctionValue(x) * f2.getFunctionValue(x);
    }
}

```

Power.java

```

package functions.meta;
import functions.Function;
public class Power implements Function {
    private final Function f;
    private final double power;

    public Power(Function f, double power) {
        this.f = f;
        this.power = power;
    }

    @Override
    public double getLeftDomainBorder() {
        return f.getLeftDomainBorder();
    }
}

```

```

    }
    @Override
    public double getRightDomainBorder() {
        return f.getRightDomainBorder();
    }
    @Override
    public double getFunctionValue(double x) {
        double value = f.getFunctionValue(x);
        return Math.pow(value, power);
    }
}

```

Scale.java

```

package functions.meta;
import functions.Function;
public class Scale implements Function {
    private final Function f;
    private final double scaleX, scaleY;
    public Scale(Function f, double scaleX, double scaleY) {
        this.f = f;
        this.scaleX = scaleX;
        this.scaleY = scaleY;
    }
    @Override
    public double getLeftDomainBorder() {
        return f.getLeftDomainBorder() * scaleX;
    }
    @Override
    public double getRightDomainBorder() {
        return f.getRightDomainBorder() * scaleX;
    }
    @Override
    public double getFunctionValue(double x) {
        return f.getFunctionValue(x / scaleX) * scaleY;
    }
}

```

Shift.java

```

package functions.meta;
import functions.Function;
public class Shift implements Function {
    private final Function f;
    private final double shiftX, shiftY;
    public Shift(Function f, double shiftX, double shiftY) {
        this.f = f;
        this.shiftX = shiftX;
        this.shiftY = shiftY;
    }
    @Override
    public double getLeftDomainBorder() {
        return f.getLeftDomainBorder() + shiftX;
    }
    @Override
    public double getRightDomainBorder() {
        return f.getRightDomainBorder() + shiftX;
    }
    @Override
    public double getFunctionValue(double x) {
        return f.getFunctionValue(x - shiftX) + shiftY;
    }
}

```

Sum.java

```

package functions.meta;
import functions.Function;
public class Sum implements Function {

```

```

private final Function f1, f2;
public Sum(Function f1, Function f2) {
    this.f1 = f1;
    this.f2 = f2;
}
@Override
public double getLeftDomainBorder() {
    return Math.max(f1.getLeftDomainBorder(), f2.getLeftDomainBorder());
}
@Override
public double getRightDomainBorder() {
    return Math.min(f1.getRightDomainBorder(),
f2.getRightDomainBorder());
}
@Override
public double getFunctionValue(double x) {
    if (x < getLeftDomainBorder() || x > getRightDomainBorder()) {
        return Double.NaN;
    }
    return f1.getFunctionValue(x) + f2.getFunctionValue(x);
}
}

```

Задание 5

В пакете functions создала класс Functions, содержащий вспомогательные статические методы для работы с функциями.

```

package functions;

import functions.meta.*;

public final class Functions {
    private Functions() {
        throw new AssertionError("Нельзя создать экземпляр утилитного класса");
    }

    public static Function shift(Function f, double shiftX, double shiftY) {
        return new Shift(f, shiftX, shiftY);
    }

    public static Function scale(Function f, double scaleX, double scaleY) {
        return new Scale(f, scaleX, scaleY);
    }

    public static Function power(Function f, double power) {
        return new Power(f, power);
    }

    public static Function sum(Function f1, Function f2) {
        return new Sum(f1, f2);
    }

    public static Function mult(Function f1, Function f2) {
        return new Mult(f1, f2);
    }

    public static Function composition(Function f1, Function f2) {
        return new Composition(f1, f2);
    }
}

```

Задание 6

В пакете functions создала класс TabulatedFunctions, содержащий вспомогательные статические методы для работы с табулированными функциями. В нем описала метод public static TabulatedFunction tabulate(Function function, double leftX, double rightX, int pointsCount), а так же сделала, чтобы он выбрасывал исключение IllegalArgumentException.

```
package functions;
import java.io.*;
public final class TabulatedFunctions {
    private TabulatedFunctions() {
        throw new AssertionError("Нельзя создать экземпляр утилитного класса");
    }
    public static TabulatedFunction tabulate(Function function, double leftX, double rightX, int pointsCount) {
        if (leftX < function.getLeftDomainBorder() || rightX > function.getRightDomainBorder()) {
            throw new IllegalArgumentException("Границы табулирования выходят за область определения функции");
        }
        FunctionPoint[] points = new FunctionPoint[pointsCount];
        double step = (rightX - leftX) / (pointsCount - 1);

        for (int i = 0; i < pointsCount; i++) {
            double x = leftX + i * step;
            double y = function.getFunctionValue(x);
            points[i] = new FunctionPoint(x, y);
        }
        return new ArrayTabulatedFunction(points); //создание таб.функции
    }
}
```

Задание 7

В класс TabulatedFunctions добавила следующие методы:

Метод вывода табулированной функции в байтовый поток public static void outputTabulatedFunction(TabulatedFunction function, OutputStream out)

Метод ввода табулированной функции из байтового потока public static TabulatedFunction inputTabulatedFunction(InputStream in)

Метод записи табулированной функции в символьный поток public static void writeTabulatedFunction(TabulatedFunction function, Writer out)

Метод чтения табулированной функции из символьного потока public static TabulatedFunction readTabulatedFunction(Reader in)

Все методы объявляют throws IOException - проверяемое исключение передается вызывающему коду. Вызывающий код сам решает, как обработать ошибки ввода-вывода.

```
public static void outputTabulatedFunction(TabulatedFunction function,
OutputStream out) throws IOException {
    DataOutputStream dos = new DataOutputStream(out);
    dos.writeInt(function.getPointsCount());

    for (int i = 0; i < function.getPointsCount(); i++) {
        dos.writeDouble(function.getPointX(i));
        dos.writeDouble(function.getPointY(i));
    }
    dos.flush(); //сбрасываем буфер
}
```

```

    }
    //метод бинарного чтения табулированной функции из потока
    public static TabulatedFunction inputTabulatedFunction(InputStream in)
    throws IOException {
        DataInputStream dis = new DataInputStream(in);
        int pointsCount = dis.readInt();
        FunctionPoint[] points = new FunctionPoint[pointsCount];

        for (int i = 0; i < pointsCount; i++) {
            double x = dis.readDouble();
            double y = dis.readDouble();
            points[i] = new FunctionPoint(x, y);
        }

        return new ArrayTabulatedFunction(points);
    }
    //запись в текстовом формате
    public static void writeTabulatedFunction(TabulatedFunction function, Writer
    out) throws IOException {
        PrintWriter writer = new PrintWriter(out);
        writer.print(function.getPointsCount());

        for (int i = 0; i < function.getPointsCount(); i++) {
            writer.print(" " + function.getPointX(i));
            writer.print(" " + function.getPointY(i));
        }
        writer.flush();
    }
    //чтение из текстового формата
    public static TabulatedFunction readTabulatedFunction(Reader in) throws
    IOException {
        StreamTokenizer tokenizer = new StreamTokenizer(in);
        tokenizer.parseNumbers();

        tokenizer.nextToken();
        int pointsCount = (int) tokenizer.nval;
        FunctionPoint[] points = new FunctionPoint[pointsCount];

        for (int i = 0; i < pointsCount; i++) {
            tokenizer.nextToken();
            double x = tokenizer.nval;
            tokenizer.nextToken();
            double y = tokenizer.nval;
            points[i] = new FunctionPoint(x, y);
        }

        return new ArrayTabulatedFunction(points);
    }
}

```

Задание 8

Создала Main.java и проверила работу функций. Изучив содержимое полученных файлов, я сделала выводы о преимуществах и недостатках каждого формата хранения. Текстовый формат удобен для отладки и ручного редактирования, но занимает больше места. Бинарный формат более компактен и быстр в работе, но не читаем человеком. Оба формата обеспечивают точное сохранение данных без потерь.

Задание 9

Реализовала сериализацию двумя способами. Оба подхода обеспечивают надежное сохранение и восстановление объектов. Serializable лучше подходит для быстрой разработки и простых случаев, тогда как Externalizable предпочтительнее когда требуется контроль над форматом данных или оптимизация производительности.

Результаты запуска Main.java

___ вывод значений синуса и косинуса на отрезке $[0, \pi]$ ___

```
sin(0,0) = 0,0000, cos(0,0) = 1,0000
sin(0,1) = 0,0998, cos(0,1) = 0,9950
sin(0,2) = 0,1987, cos(0,2) = 0,9801
sin(0,3) = 0,2955, cos(0,3) = 0,9553
sin(0,4) = 0,3894, cos(0,4) = 0,9211
sin(0,5) = 0,4794, cos(0,5) = 0,8776
sin(0,6) = 0,5646, cos(0,6) = 0,8253
sin(0,7) = 0,6442, cos(0,7) = 0,7648
sin(0,8) = 0,7174, cos(0,8) = 0,6967
sin(0,9) = 0,7833, cos(0,9) = 0,6216
sin(1,0) = 0,8415, cos(1,0) = 0,5403
sin(1,1) = 0,8912, cos(1,1) = 0,4536
sin(1,2) = 0,9320, cos(1,2) = 0,3624
sin(1,3) = 0,9636, cos(1,3) = 0,2675
sin(1,4) = 0,9854, cos(1,4) = 0,1700
sin(1,5) = 0,9975, cos(1,5) = 0,0707
sin(1,6) = 0,9996, cos(1,6) = -0,0292
sin(1,7) = 0,9917, cos(1,7) = -0,1288
sin(1,8) = 0,9738, cos(1,8) = -0,2272
sin(1,9) = 0,9463, cos(1,9) = -0,3233
sin(2,0) = 0,9093, cos(2,0) = -0,4161
sin(2,1) = 0,8632, cos(2,1) = -0,5048
sin(2,2) = 0,8085, cos(2,2) = -0,5885
sin(2,3) = 0,7457, cos(2,3) = -0,6663
sin(2,4) = 0,6755, cos(2,4) = -0,7374
sin(2,5) = 0,5985, cos(2,5) = -0,8011
sin(2,6) = 0,5155, cos(2,6) = -0,8569
sin(2,7) = 0,4274, cos(2,7) = -0,9041
sin(2,8) = 0,3350, cos(2,8) = -0,9422
sin(2,9) = 0,2392, cos(2,9) = -0,9710
sin(3,0) = 0,1411, cos(3,0) = -0,9900
sin(3,1) = 0,0416, cos(3,1) = -0,9991
```

___ тестирование табулированных функций ___

--- сравнение аналитического и табулированного синуса ---

```
x=0,0: аналитический=0,0000, табулированный=0,0000
x=0,1: аналитический=0,0998, табулированный=0,0980
x=0,2: аналитический=0,1987, табулированный=0,1960
x=0,3: аналитический=0,2955, табулированный=0,2939
x=0,4: аналитический=0,3894, табулированный=0,3859
x=0,5: аналитический=0,4794, табулированный=0,4721
x=0,6: аналитический=0,5646, табулированный=0,5582
x=0,7: аналитический=0,6442, табулированный=0,6440
x=0,8: аналитический=0,7174, табулированный=0,7079
x=0,9: аналитический=0,7833, табулированный=0,7719
x=1,0: аналитический=0,8415, табулированный=0,8358
x=1,1: аналитический=0,8912, табулированный=0,8840
x=1,2: аналитический=0,9320, табулированный=0,9180
x=1,3: аналитический=0,9636, табулированный=0,9521
x=1,4: аналитический=0,9854, табулированный=0,9848
x=1,5: аналитический=0,9975, табулированный=0,9848
x=1,6: аналитический=0,9996, табулированный=0,9848
x=1,7: аналитический=0,9917, табулированный=0,9848
x=1,8: аналитический=0,9738, табулированный=0,9662
x=1,9: аналитический=0,9463, табулированный=0,9322
x=2,0: аналитический=0,9093, табулированный=0,8981
```

x=2,1: аналитический=0,8632, табулированный=0,8624
x=2,2: аналитический=0,8085, табулированный=0,7985
x=2,3: аналитический=0,7457, табулированный=0,7345
x=2,4: аналитический=0,6755, табулированный=0,6706
x=2,5: аналитический=0,5985, табулированный=0,5941
x=2,6: аналитический=0,5155, табулированный=0,5079
x=2,7: аналитический=0,4274, табулированный=0,4217
x=2,8: аналитический=0,3350, табулированный=0,3347
x=2,9: аналитический=0,2392, табулированный=0,2367
x=3,0: аналитический=0,1411, табулированный=0,1387
x=3,1: аналитический=0,0416, табулированный=0,0408

--- проверка экспоненты в текстовом формате ---
сравнение экспоненты до и после текстовой сериализации:

x=0,0: original=1,0000, loaded=1,0000
x=1,0: original=2,7183, loaded=2,7183
x=2,0: original=7,3891, loaded=7,3891
x=3,0: original=20,0855, loaded=20,0855
x=4,0: original=54,5982, loaded=54,5982
x=5,0: original=148,4132, loaded=148,4132
x=6,0: original=403,4288, loaded=403,4288
x=7,0: original=1096,6332, loaded=1096,6332
x=8,0: original=2980,9580, loaded=2980,9580
x=9,0: original=8103,0839, loaded=8103,0839
x=10,0: original=22026,4658, loaded=22026,4658

--- проверка текстового формата для функции cos ---
сравнение исходной и восстановленной функции:

x= 0,0: original= 1,0000, loaded= 1,0000
x= 0,3: original= 0,9397, loaded= 0,9397
x= 0,7: original= 0,7660, loaded= 0,7660
x= 1,0: original= 0,5000, loaded= 0,5000
x= 1,4: original= 0,1736, loaded= 0,1736
x= 1,7: original=-0,1736, loaded=-0,1736
x= 2,1: original=-0,5000, loaded=-0,5000
x= 2,4: original=-0,7660, loaded=-0,7660
x= 2,8: original=-0,9397, loaded=-0,9397
x= 3,1: original=-1,0000, loaded=-1,0000

--- проверка бинарного формата для функции sin ---
сравнение исходной и восстановленной функции:

x= 0,0: original= 0,0000, loaded= 0,0000
x= 0,3: original= 0,3420, loaded= 0,3420
x= 0,7: original= 0,6428, loaded= 0,6428
x= 1,0: original= 0,8660, loaded= 0,8660
x= 1,4: original= 0,9848, loaded= 0,9848
x= 1,7: original= 0,9848, loaded= 0,9848
x= 2,1: original= 0,8660, loaded= 0,8660
x= 2,4: original= 0,6428, loaded= 0,6428
x= 2,8: original= 0,3420, loaded= 0,3420
x= 3,1: original= 0,0000, loaded= 0,0000

--- проверка логарифма в бинарном формате ---
сравнение логарифма до и после бинарной сериализации:

x=1,0: original=0,0000, loaded=0,0000
x=2,0: original=0,6931, loaded=0,6931
x=3,0: original=1,0986, loaded=1,0986
x=4,0: original=1,3863, loaded=1,3863
x=5,0: original=1,6094, loaded=1,6094
x=6,0: original=1,7918, loaded=1,7918
x=7,0: original=1,9459, loaded=1,9459
x=8,0: original=2,0794, loaded=2,0794
x=9,0: original=2,1972, loaded=2,1972
x=10,0: original=2,3026, loaded=2,3026

___ тестирование комбинаций функций ___
проверка тождества $\sin^2(x) + \cos^2(x) \equiv 1$:
 $\sin^2(0,0) + \cos^2(0,0) = 1,0000$
 $\sin^2(0,1) + \cos^2(0,1) = 1,0000$

$\sin^2(0,2) + \cos^2(0,2) = 1,0000$
 $\sin^2(0,3) + \cos^2(0,3) = 1,0000$
 $\sin^2(0,4) + \cos^2(0,4) = 1,0000$
 $\sin^2(0,5) + \cos^2(0,5) = 1,0000$
 $\sin^2(0,6) + \cos^2(0,6) = 1,0000$
 $\sin^2(0,7) + \cos^2(0,7) = 1,0000$
 $\sin^2(0,8) + \cos^2(0,8) = 1,0000$
 $\sin^2(0,9) + \cos^2(0,9) = 1,0000$
 $\sin^2(1,0) + \cos^2(1,0) = 1,0000$
 $\sin^2(1,1) + \cos^2(1,1) = 1,0000$
 $\sin^2(1,2) + \cos^2(1,2) = 1,0000$
 $\sin^2(1,3) + \cos^2(1,3) = 1,0000$
 $\sin^2(1,4) + \cos^2(1,4) = 1,0000$
 $\sin^2(1,5) + \cos^2(1,5) = 1,0000$
 $\sin^2(1,6) + \cos^2(1,6) = 1,0000$
 $\sin^2(1,7) + \cos^2(1,7) = 1,0000$
 $\sin^2(1,8) + \cos^2(1,8) = 1,0000$
 $\sin^2(1,9) + \cos^2(1,9) = 1,0000$
 $\sin^2(2,0) + \cos^2(2,0) = 1,0000$
 $\sin^2(2,1) + \cos^2(2,1) = 1,0000$
 $\sin^2(2,2) + \cos^2(2,2) = 1,0000$
 $\sin^2(2,3) + \cos^2(2,3) = 1,0000$
 $\sin^2(2,4) + \cos^2(2,4) = 1,0000$
 $\sin^2(2,5) + \cos^2(2,5) = 1,0000$
 $\sin^2(2,6) + \cos^2(2,6) = 1,0000$
 $\sin^2(2,7) + \cos^2(2,7) = 1,0000$
 $\sin^2(2,8) + \cos^2(2,8) = 1,0000$
 $\sin^2(2,9) + \cos^2(2,9) = 1,0000$
 $\sin^2(3,0) + \cos^2(3,0) = 1,0000$
 $\sin^2(3,1) + \cos^2(3,1) = 1,0000$

=== тестирование сериализации ===

исходная функция ($\log(\exp(x))$):

$x=0,0: y=\text{NaN}$
 $x=1,0: y=1,0000$
 $x=2,0: y=2,0000$
 $x=3,0: y=3,0000$
 $x=4,0: y=4,0000$
 $x=5,0: y=5,0000$
 $x=6,0: y=6,0000$
 $x=7,0: y=7,0000$
 $x=8,0: y=8,0000$
 $x=9,0: y=9,0000$
 $x=10,0: y=10,0000$

сравнение после serializable:

сравнение исходной и восстановленной функции:

$x= 0,0: \text{original}= \text{NaN}, \text{loaded}= \text{NaN}$
 $x= 1,0: \text{original}= 1,0000, \text{loaded}= 1,0000$
 $x= 2,0: \text{original}= 2,0000, \text{loaded}= 2,0000$
 $x= 3,0: \text{original}= 3,0000, \text{loaded}= 3,0000$
 $x= 4,0: \text{original}= 4,0000, \text{loaded}= 4,0000$
 $x= 5,0: \text{original}= 5,0000, \text{loaded}= 5,0000$
 $x= 6,0: \text{original}= 6,0000, \text{loaded}= 6,0000$
 $x= 7,0: \text{original}= 7,0000, \text{loaded}= 7,0000$
 $x= 8,0: \text{original}= 8,0000, \text{loaded}= 8,0000$
 $x= 9,0: \text{original}= 9,0000, \text{loaded}= 9,0000$
 $x=10,0: \text{original}=10,0000, \text{loaded}=10,0000$

сравнение после externalizable:

сравнение исходной и восстановленной функции:

$x= 0,0: \text{original}= \text{NaN}, \text{loaded}= \text{NaN}$
 $x= 1,0: \text{original}= 1,0000, \text{loaded}= 1,0000$
 $x= 2,0: \text{original}= 2,0000, \text{loaded}= 2,0000$
 $x= 3,0: \text{original}= 3,0000, \text{loaded}= 3,0000$
 $x= 4,0: \text{original}= 4,0000, \text{loaded}= 4,0000$
 $x= 5,0: \text{original}= 5,0000, \text{loaded}= 5,0000$
 $x= 6,0: \text{original}= 6,0000, \text{loaded}= 6,0000$
 $x= 7,0: \text{original}= 7,0000, \text{loaded}= 7,0000$

x= 8,0: original= 8,0000, loaded= 8,0000
x= 9,0: original= 9,0000, loaded= 9,0000
x=10,0: original=10,0000, loaded=10,0000